Session 25 — Study Guide

Status: PREPARED DRAFT

What we learned today

1. **Server Actions as the real mutation boundary** — a plain `async` function marked `"use server"` that the client calls like a function, but that actually runs on the real server, where it is allowed to read/write cookies.
2. **A real httpOnly session cookie** — the backend's own JWT, stored where browser JavaScript can never read it, set and cleared only from Server Actions.
3. **A real protected-route layout** — one session check, in one place, inherited by every page nested beneath it.
4. **Centralized 401 handling** — a stale session is detected once, in one place, and includes the real Next.js restriction that cookies cannot be mutated during a Server Component's own render.
5. **Redux Toolkit, for one real justified case** — a current-user snapshot a Client Component cannot itself fetch, never workspace/issue/project data.

Why we needed it

Session 24 gave the app a real visual design and real filtering, but the app still had no concept of a signed-in user at all — every page was equally visible to everyone, and nothing in the app could remember who was logged in between requests. Today builds the real mechanism that changes that: a real login, a real secret kept away from the browser, and a real way to protect every page that follows.

First-Time Tool Box

Tool: Redux Toolkit (`@reduxjs/toolkit` + `react-redux`, this cohort's genuine first real use).

What it is: a state-management library, used here for exactly one real purpose: holding the signed-in user's own name/email so a Client Component (the account bar) can display it without itself being able to `await` the backend.

Why we use it here: a Server Component can fetch the real current user directly; a Client Component cannot. Redux is the real bridge between the two, used deliberately narrowly — never for workspace/issue/project data, which stays real server data, fetched fresh.

Where/how you run it: `npm install @reduxjs/toolkit react-redux`; then `createSlice` (the data + its own reducer), `configureStore` (wrapped in a real per-request factory function, `makeStore()`), and a `Provider` inside a `"use client"` bridge component.

Commands/actions needed today:

1. `npm install @reduxjs/toolkit react-redux`.
2. Writing `currentUserSlice.ts` (one reducer, `setCurrentUser`).
3. Writing `store.ts`'s own `makeStore()` factory.

4. Writing `StoreProvider.tsx`, hydrating the store once with a value a Server Component already fetched.

How to tell it worked: a Client Component using `useSelector` shows the real signed-in user's own name, without ever calling the backend itself.

Common first-time mistakes: exporting one shared `configureStore(...)` call instead of a factory function — a real concurrency bug, since a real server handles many requests at once.

The problem, before the solution

Before today, the app had no login at all. Even if it had, there was nowhere safe to keep the result of a successful login between one request and the next, without exposing it to any script running in the browser — including a future, accidental XSS bug.

The solution, step by step

A real httpOnly session cookie. WHAT IS IT: a cookie set with Next's own `cookies()` API, with `httpOnly: true`. WHY: `httpOnly` makes the cookie invisible to `document.cookie` and every other client-side read — the browser's own network layer still sends it automatically, but no script can read it. WHAT BREAKS WITHOUT IT: the real token would be readable by any script on the page, including one injected via a future XSS bug. HOW: `store.set("session", token, { httpOnly: true, secure: ..., sameSite: "lax", ... })`. WHAT TO LOOK AT: `src/lib/session.ts`. WHAT TO EXPECT: a real cookie named `session` appears in DevTools, with no way to read its value from the browser's own JS console.

A real Server Action as the mutation boundary. WHAT IS IT: a function in a file (or marked inline) with `"use server"`, called from a `<form action={...}>` or directly. WHY: it is the one real place client code can trigger server-only work (like setting that cookie) without ever exposing a client-visible API route to do it. WHAT BREAKS WITHOUT IT: nothing could ever set the real session cookie from a client interaction at all — Server Components can read cookies, but only a Server Action/Route Handler can write them. HOW: `"use server"; export async function loginAction(prevState, formData) { ... }`. WHAT TO LOOK AT: `src/lib/actions/auth.ts`. WHAT TO EXPECT: submitting the real login form actually sets the session cookie and redirects.

A real protected-route layout. WHAT IS IT: a layout component that calls `requireSessionToken()` before rendering `children`. WHY: one real check, in one real place, inherited by every nested route — never repeated per-page. WHAT BREAKS WITHOUT IT: every individual protected page would need to repeat the same check, and the one time it's forgotten is a real security hole. HOW: `const token = await requireSessionToken(); return children;` WHAT TO LOOK AT: `src/app/(app)/layout.tsx`. WHAT TO EXPECT: visiting any page under `(app)` with no real session redirects to `/login`.

Centralized 401 handling. WHAT IS IT: a single check inside `apiFetch` — if a request that DID carry a token gets a real `401` back, the session is stale. WHY: the real backend is the one authority on whether a token is still valid; a stale session should clear itself and send the user back to login, not show a generic error. WHAT BREAKS WITHOUT IT: an expired session would surface as a confusing, unhandled error on whatever page the user happened to be on. HOW: `if (res.status === 401 && token) { try { await clearSessionToken(); } catch {} ; redirect("/login"); }` — the `try/catch` exists because Next.js

forbids mutating cookies during a Server Component's own render; the failure there is expected and swallowed. WHAT TO LOOK AT: `src/lib/api.ts`. WHAT TO EXPECT: a garbage/expired session cookie produces a clean redirect to `/login`, never a server error page.

Connecting to earlier sessions

The real backend's own JWT and login endpoint already existed (Session 14/15) — today connects the frontend to that already-real backend, the same "the backend already does this, the frontend finally uses it" pattern Session 24 used for the backend's own `status` / `title` filters.

Glossary (required)

- **Server Action:** a function marked `"use server"` that runs on the server but can be called from client code as if it were a plain function.
- **httpOnly cookie:** a cookie flagged so that no client-side JavaScript (first-party or injected) can ever read its value via `document.cookie`.
- **Protected layout:** a layout component whose own real session check every nested route inherits automatically.
- **Current-user snapshot:** a small, display-only piece of Redux state, populated FROM an already-authenticated request — never itself an authentication mechanism.

Common mistakes

- Passing `loginAction` directly to `<form action={loginAction}>` instead of the real `formAction` that `useActionState` returns.
- Checking `res.status === 401` without the `&& token` guard — fires on a failed login attempt too, redirecting away from the very page showing the "wrong password" error.
- Treating Redux's `currentUserSlice` as the thing that performs authentication, instead of a display convenience populated after a real check already succeeded elsewhere.

Common errors and how to debug them

- **"Cookies can only be modified in a Server Action or Route Handler":** expected, during a Server Component's own render, the first time an expired session is hit — this is why the real clear is wrapped in a `try/catch`; never remove that wrapper to "fix" the error.
- **A protected page renders with no real user data:** confirm the layout's own `apiFetch("/auth/me", { token })` call actually passed the token — a missing token means the real route (`JwtAuthGuard`-protected) returns its own `401`.
- **The account bar never shows a name:** confirm `AccountBar` reads `useSelector((s) => s.currentUser.user)`, not the whole slice object.

Never "fix" any of these by removing the real server-side check and trusting the client instead.

Review questions (required)

1. Why is the real session token stored in an httpOnly cookie instead of `localStorage`?
2. Why does `apiFetch`'s own 401 check require `&& token`, not just `res.status === 401`?
3. Why can only a Server Action (not a Server Component's own render) mutate cookies?
4. What real problem does `currentUserSlice` solve that the protected layout's own check does not already solve by itself?

Answers

1. An httpOnly cookie is invisible to any client-side JavaScript, including a script injected via a future XSS bug; `localStorage` has no such protection.
2. Without it, a real failed login attempt (wrong password, no token ever sent) would also trigger a session-clearing redirect, hiding the real "wrong password" error.
3. This is a real, deliberate Next.js framework restriction — Server Components may only read data during render, never cause side effects like cookie mutation; only a Server Action or Route Handler runs outside that render cycle.
4. The protected layout's check answers "is this request authenticated at all" — it never answers "what does a Client Component, which cannot `await` the backend, actually render for this user's own name right now." Those are genuinely different questions.

What you should be able to explain now (required)

- Why this app never lets browser JavaScript read the real session token, and what mechanism specifically enforces that.
- Why a protected-route layout is placed where it is, and what every route nested beneath it gets "for free."
- Why Redux's current-user snapshot is a display convenience, never itself part of the real authentication check.

Not yet (required)

- Real Issue create/edit/delete, drag-and-drop board, Sprint/Backlog (Session 26).
- `@mention` autocomplete, Activity feed, Notifications UI (Session 27).
- Server-side role/permission UI gating beyond the owner-protection already enforced server-side (Session 26).